CHAPTER 1

Your First Engineering Program: Solving a Quadratic Equation with MATLAB and AI

Learning Objectives

After completing this chapter, you should be able to:

- translate a mathematical problem into computational steps;
- construct a simple flowchart;
- write and run a MATLAB script;
- use variables, arithmetic operators, and built-in functions;
- use `if-elseif-else` to make logical decisions;
- solve a quadratic equation using your own algorithm;
- solve the same equation using MATLAB's `roots()` function;
- visualize a quadratic function;
- use AI to improve a MATLAB program;
- verify computational results; and
- distinguish between a mathematically correct solution and a physically meaningful engineering solution.

1.1 Why Learn Programming When We Have AI?

AI can generate MATLAB code within seconds.

MATLAB itself already contains thousands of functions capable of performing calculations that engineers once had to program themselves.

So why should an engineering student still learn programming?

Because an engineer must still be able to:

- understand the problem;
- formulate the mathematics;
- develop computational logic;
- recognize incorrect results;
- verify calculations;
- interpret physical meaning; and
- make engineering decisions.

AI can write code. The engineer remains responsible for the solution.

Throughout this book, we will use the following approach:

PROBLEM → THINK → BUILD → RUN → ASK AI → IMPROVE → VERIFY → INTERPRET

We will therefore not learn MATLAB by memorizing commands one after another.

We will **start with problems**.

MATLAB commands will appear naturally when we need them.

1.2 Our First Problem: The Quadratic Equation

Consider

$$ax^2 + bx + c = 0$$

The familiar quadratic formula is

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Let us begin with

$$x^2 - 5x + 6 = 0$$

Therefore,

$$a = 1,\ b = -5,\ c = 6$$

Before opening MATLAB, think about the problem.

INPUT

$$a,\ b,\ c$$

PROCESS

Calculate

$$D = b^2 - 4ac$$

and then

$$x_1 = \frac{-b + \sqrt{D}}{2a}$$
$$x_2 = \frac{-b - \sqrt{D}}{2a}$$

OUTPUT

$$x_1,\ x_2$$

We have already developed an **algorithm** without writing a single line of MATLAB code.

Good programming begins with thinking, not typing.

1.3 From Algorithm to Flowchart

A flowchart represents computational logic graphically.

Standard symbols include:

- **ellipse** — start/end;
- **parallelogram** — input/output;
- **rectangle** — computational process;
- **diamond** — logical decision;
- **arrow** — direction of program flow.

For our quadratic-equation solver, the logic is:

START

↓

Input a, b, c

↓

◇ **Is** $a = 0$**?**

YES → the equation is not quadratic → solve $bx + c = 0$

NO ↓

Calculate

$$D = b^2 - 4ac$$

↓

◇ **Is $D > 0$?**

YES → two distinct real roots

NO ↓

◇ **Is $D = 0$?**

YES → one repeated real root

NO → two complex conjugate roots

↓

Display solution(s)

↓

END

Think

Why must we check whether

$$a = 0$$

before applying the quadratic formula?

What would happen to

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

if $a = 0$?

1.4 Build the Simplest Program First

Do not ask AI yet.

Open MATLAB and create a new script.

Type:

```
a = 1;
b = -5;
```

```
c = 6;

D = b^2 - 4*a*c;

x1 = (-b + sqrt(D))/(2*a);
x2 = (-b - sqrt(D))/(2*a);

disp(x1)
disp(x2)
```

Run the program.

MATLAB should return

$$x_1 = 3,\ x_2 = 2$$

Congratulations.

You have written your first MATLAB program.

1.5 What Did We Just Learn?

We did not begin with a lecture about MATLAB commands.

Yet we have already encountered several programming concepts.

Variables

```
a = 1;
```

`a` is a variable.

So are

```
b
c
D
x1
x2
```

Arithmetic operations

```
D = b^2 - 4*a*c;
```

MATLAB recognizes familiar mathematical operators:

```
+     addition
-     subtraction
```

```
*     multiplication
/     division
^     power
```

Built-in mathematical functions

`sqrt(D)`

`sqrt()` is a MATLAB built-in function.

Displaying results

`disp(x1)`

asks MATLAB to display the value of `x1`.

Experiment

Remove the semicolon from:

`a = 1;`

Run the program again.

What changes?

Put the semicolon back.

What do you think ; does?

1.6 Experiment Before Moving On

Change the equation to

$$x^2 - 7x + 12 = 0$$

Change only:

```
a = ?
b = ?
c = ?
```

Before pressing **Run**:

Predict

What should the roots be?

Run

Execute the program.

Verify

Substitute the calculated roots into

$$x^2 - 7x + 12$$

Do you obtain zero?

Prediction before computation is an excellent engineering habit.

1.7 Can We Trust the Computer?

Return to

$$x^2 - 5x + 6 = 0$$

MATLAB gives

$$x = 3$$

Check manually:

$$\begin{aligned} &3^2 - 5(3) + 6 \\ &= 9 - 15 + 6 \\ &= 0 \end{aligned}$$

Therefore $x = 3$satisfies the equation.

Now verify $x = 2$.

This simple habit becomes increasingly important as our programs become more complicated.

It becomes **even more important when AI writes the program for us**.

Never accept a result merely because a computer produced it.

1.8 The Discriminant Changes Everything

Consider

$$D = b^2 - 4ac$$

The discriminant determines the nature of the roots.

Case 1

$$D > 0$$

There are **two distinct real roots**.

Case 2

$$D = 0$$

There is **one repeated real root**.

Case 3

$$D < 0$$

There are **two complex conjugate roots**.

Try the following equations.

Experiment A

$$x^2 - 5x + 6 = 0$$

Experiment B

$$x^2 - 4x + 4 = 0$$

Experiment C

$$x^2 + 2x + 5 = 0$$

Before running MATLAB, predict whether each equation has:

- two real roots;
- one repeated root; or
- complex roots.

1.9 Teaching the Computer to Make Decisions

Our first program performs calculations.

Now we want it to make a **logical decision**.

MATLAB provides:

```
if
    ...
elseif
    ...
else
    ...
end
```

The logic is:

```
IF D > 0

    calculate two real roots

ELSE IF D = 0

    calculate one repeated root

ELSE

    calculate two complex roots
```

Try constructing this program yourself before looking at the complete version.

1.10 Complete Flowchart-Based MATLAB Program

Now we translate the complete flowchart into MATLAB.

```
% ========================================================
% QUADRATIC EQUATION SOLVER
% Solves:
%
%       a*x^2 + b*x + c = 0
%
% ========================================================

clc;
```

```
clear;

disp('========================================')
disp('       QUADRATIC EQUATION SOLVER')
disp('========================================')


%% STEP 1 - INPUT

a = input('Enter coefficient a: ');
b = input('Enter coefficient b: ');
c = input('Enter coefficient c: ');


%% STEP 2 - IS THE EQUATION QUADRATIC?

if a == 0

    disp('This is NOT a quadratic equation.')

    % Equation becomes:
    %
    %       b*x + c = 0

    if b ~= 0

        x = -c/b;

        fprintf('The equation is linear.\n');
        fprintf('x = %.6f\n',x);

    else

        if c == 0

            disp('Infinitely many solutions.')

        else

            disp('No solution.')

        end

    end


else

    %% STEP 3 - CALCULATE DISCRIMINANT

    D = b^2 - 4*a*c;

    fprintf('\nDiscriminant D = %.6f\n',D);


    %% STEP 4 - DETERMINE THE ROOTS

    if D > 0

        disp('Two distinct real roots.')
```

```
        x1 = (-b + sqrt(D))/(2*a);
        x2 = (-b - sqrt(D))/(2*a);

        fprintf('x1 = %.6f\n',x1);
        fprintf('x2 = %.6f\n',x2);


    elseif D == 0

        disp('One repeated real root.')

        x = -b/(2*a);

        fprintf('x = %.6f\n',x);


    else

        disp('Two complex conjugate roots.')

        realPart = -b/(2*a);
        imagPart = sqrt(-D)/(2*a);

        x1 = realPart + 1i*imagPart;
        x2 = realPart - 1i*imagPart;

        fprintf('x1 = %.6f %+.6fi\n', ...
            real(x1),imag(x1));

        fprintf('x2 = %.6f %+.6fi\n', ...
            real(x2),imag(x2));

    end

end


%% END

disp(' ')
disp('========================================')
disp('              END OF PROGRAM')
disp('========================================')
```

1.11 Flowchart and MATLAB: Can You See the Connection?

Compare the program with the flowchart.

Flowchart	MATLAB
START	Start script
Input a, b, c	`input()`
Is $a = 0$?	`if a == 0`
Solve linear equation	`x = -c/b`

Flowchart	MATLAB
Calculate D	`D = b^2 - 4*a*c`
Is $D > 0$?	`if D > 0`
Is $D = 0$?	`elseif D == 0`
Otherwise	`else`
Display	`disp()`, `fprintf()`
END	End script

The MATLAB syntax is new.

The **logic is not**.

We developed the logic before writing the program.

1.12 MATLAB Can Already Solve the Equation!

There is something interesting.

MATLAB already has a built-in function for calculating polynomial roots:

```
roots()
```

For

$$x^2 - 5x + 6 = 0$$

write:

```
p = [1 -5 6];
x = roots(p)
```

MATLAB interprets

```
[1 -5 6]
```

as the coefficients of

$$x^2 - 5x + 6.$$

We can make it even shorter:

```
x = roots([1 -5 6])
```

Or, using variables:

```
a = 1;
b = -5;
c = 6;

x = roots([a b c])
```

Compare the results with our previous program.

They should agree.

1.13 Then Why Did We Write the Long Program?

Excellent question.

If MATLAB can solve the problem using

```
roots([a b c])
```

why did we write dozens of lines of code?

Because:

Using a function is not the same as understanding the computation.

By developing the algorithm ourselves, we encountered:

- variables;
- mathematical operators;
- built-in functions;
- program sequence;
- logical decisions;
- `if`;
- `elseif`;
- `else`;
- input;
- output;
- verification; and
- computational thinking.

The built-in function makes us **faster**.

Understanding the algorithm makes us **better problem solvers**.

1.14 Make MATLAB Draw the Mathematics

Consider again

$$y = x^2 - 5x + 6$$

Try:

```
x = -2:0.1:7;

y = x.^2 - 5*x + 6;

plot(x,y)

grid on

xlabel('x')
ylabel('y')

title('Quadratic Function')
```

Look carefully at the graph.

Questions

Where does the curve cross the x-axis?

Compare these locations with

$$x_1 = 3,\ x_2 = 2.$$

The graph and the numerical calculation are describing the same mathematical problem.

1.15 Now Make It an Engineering Problem

Quadratic equations are not merely school mathematics.

They can arise in:

- motion and trajectories;
- structural relationships;
- hydraulic problems;
- geometric design;
- optimization;
- intersection problems; and
- many engineering models.

Consider a simple civil-engineering example.

Object Released Above a Water Surface

Suppose an object is projected upward from a bridge deck located

$$20 \text{ m}$$

above the water surface.

Its initial upward velocity is

$$v_0 = 10 \text{ m/s}.$$

The vertical position is

$$z = z_0 + v_0 t - \frac{1}{2} g t^2$$

where

$$g = 9.81 \text{ m/s}^2.$$

At the water surface,

$$z = 0.$$

Therefore,

$$0 = 20 + 10t - \frac{1}{2}(9.81)t^2$$

or

$$4.905t^2 - 10t - 20 = 0.$$

Therefore,

$$a = 4.905,\ b = -10,\ c = -20.$$

1.16 Think Before Running MATLAB

Before calculating anything:

Predict

Will the equation have one or two mathematical roots?

Analyze

What does $t$represent physically?

Evaluate

If MATLAB produces a negative value of t, does that value have physical meaning for the event occurring **after** $t = 0$?

Now calculate:

```
a = 4.905;
b = -10;
c = -20;

t = roots([a b c])
```

MATLAB gives two mathematical solutions.

But only one corresponds to the time **after the object is projected**.

This leads to a fundamental engineering principle:

A mathematically correct answer is not necessarily a physically meaningful engineering answer.

MATLAB solves the mathematics.

The engineer interprets the physics.

1.17 Now Invite AI Into the Classroom

Until now, we have developed the program ourselves.

Now AI becomes our programming partner.

Give an AI assistant your simple program and ask:

Improve this MATLAB program for solving a quadratic equation. Make it robust and user-friendly. Handle real and complex roots, check invalid input, plot the quadratic function, identify the roots and vertex, and explain your major improvements.

Run the AI-generated program.

But do not simply copy it.

Study it.

1.18 Question the AI

Ask yourself:

1. What did AI change?
2. Why did it make those changes?
3. What unfamiliar MATLAB commands did it introduce?
4. Are all those commands necessary?
5. Is the program easier or harder to understand?
6. Does it give the correct roots?
7. Does the graph agree with the numerical results?
8. Did AI make any unnecessary assumptions?

AI-generated code should be examined, not worshipped.

1.19 Four Ways to Solve One Problem

We can now solve the quadratic equation in four different ways.

Method 1 — Mathematics

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Method 2 — Our MATLAB Algorithm

We explicitly program the quadratic formula and decision logic.

Method 3 — MATLAB Built-In Function

```
x = roots([a b c]);
```

Method 4 — AI-Assisted Programming

Describe the problem and ask AI to develop or improve the MATLAB program.

Which method is best?

There is no universal answer.

The engineer must decide which approach is appropriate.

1.20 Human vs MATLAB vs AI

Think about the different roles.

HUMAN

- identifies the problem;
- understands the physical system;
- formulates assumptions;
- develops or evaluates the logic;
- verifies results;
- interprets meaning;
- makes decisions.

MATLAB

- performs computation;
- processes data;
- solves equations;
- visualizes results;
- executes algorithms rapidly.

AI

- generates code;
- explains code;
- suggests algorithms;
- identifies errors;
- improves programs;
- proposes alternative approaches.

Therefore:

Human builds and evaluates the logic. MATLAB performs the computation. AI expands the possibilities. The engineer remains responsible for the result.

1.21 Higher-Order Thinking

Programming should not stop at:

“Can you reproduce the code?”

We should move further.

ANALYZE

Why does the discriminant determine the nature of the roots?

EVALUATE

Which is better?

Program A: 15 lines that you completely understand.

Program B: 100 sophisticated AI-generated lines that you only partly understand.

Explain your reasoning.

CREATE

Develop your own quadratic-equation solver that:

- accepts user input;
- checks whether the equation is quadratic;
- identifies the nature of the roots;
- calculates the roots;
- plots the quadratic function;
- marks the roots;
- identifies the vertex; and
- presents the results clearly.

You may use AI.

But:

You must be able to explain and defend your program.

1.22 Engineering Challenge

The bridge problem gave us the equation directly.

Real engineering problems usually do not.

Create or find a civil-engineering problem involving a quadratic relationship.

Your task is to:

1. describe the physical problem;

2. identify the known variables;
3. identify the unknown;
4. formulate the quadratic equation;
5. draw the computational flowchart;
6. solve it using your own MATLAB algorithm;
7. solve it using `roots()`;
8. compare the results;
9. ask AI for an alternative solution;
10. verify the answer; and
11. explain which solution is physically meaningful.

This is much closer to what engineers actually do.

1.23 What Have We Learned?

From one simple quadratic equation, we encountered a surprising amount of material.

MATHEMATICS

- quadratic equations;
- discriminant;
- real roots;
- complex roots.

COMPUTATIONAL THINKING

- input;
- process;
- output;
- algorithms;
- flowcharts;
- decisions.

MATLAB

```
variables
sqrt()
disp()
fprintf()
input()
if
elseif
else
roots()
plot()
```

AI

- code generation;
- code explanation;
- debugging;
- program improvement;
- alternative solutions.

ENGINEERING THINKING

Predict → Calculate → Visualize → Verify → Interpret → Question → Create

---

1.24 The Most Important Lesson

The most important lesson in this chapter is not the quadratic formula.

It is not `if`.

It is not `roots()`.

It is not even MATLAB.

The central lesson is:

UNDERSTAND THE PROBLEM

↓

FORMULATE THE MATHEMATICS

↓

DESIGN THE LOGIC

↓

BUILD A SIMPLE PROGRAM

↓

RUN AND DEBUG

↓

USE MATLAB'S POWER

↓

USE AI TO EXTEND THE SOLUTION

↓

VERIFY

↓

INTERPRET AS AN ENGINEER

The computer is extraordinarily powerful.

AI makes it even more powerful.

But engineering judgment remains human responsibility.

CURIOSITY BOX — WHAT COMES NEXT?

Our first program deals with only three input numbers:

$$a,\ b,\ c$$

But civil engineers rarely work with only three numbers.

What if we have:

- 100 concrete-strength measurements?
- 1,000 survey observations?
- 30 years of rainfall records?
- hourly water levels?
- thousands of traffic observations?
- millions of sensor measurements?

Should we create one variable for every number?

Obviously not.

We need a better way to organize, manipulate, and analyze data.

That brings us to:

CHAPTER 2

From One Number to Thousands: Vectors, Matrices, and Engineering Data